

数据结构

夏天

`xiat@ruc.edu.cn`

中国人民大学信息资源管理学院

字符串

计算机非数值处理的对象经常是字符串数据。如信息检索、文本聚类、文本理解、搜索引擎、敏感词过滤、生物序列分析等。

- 串的定义及表示
- 串的存储结构
- 串的模式匹配（核心重点）
- 经典匹配算法：BF、KMP、AC 自动机
- 实战应用：Python 高效实现

串 (String) 的定义

- 串是由零个或多个任意字符组成的字符序列。它也是一种特殊的线性表，其数据元素仅由一个字符组成，而且它通常是作为一个整体来处理。

例：s="Renmin University", s 是串名，Renmin University 为串值，其中一个一个的字符称为串的元素。

- 基本概念：
 - 空串：长度为 0 的串
 - 子串：串中任意连续字符组成的序列
 - 主串：包含子串的串
 - 模式串：用于匹配查找的目标子串
- 请思考如何实现上述串的存储？

字符串的存储

- 顺序存储: 用一组地址连续的存储单元存储串值中的字符序列。
 - 优点: 随机访问快
 - 缺点: 插入、删除效率低
- 链式存储: 考虑到存储密度, 可以按块分配。
 - 优点: 插入、删除灵活
 - 缺点: 存储密度低, 访问效率低
- 高级语言封装:
 - 在 Python/C/Java 等语言中, String 就是字符串。
 - 在 C 语言中没有 string 类型变量, 字符串用字符数组表示。
 - 现代语言均对字符串做了优化, 兼顾效率与易用性

串的模式匹配

- 设 s 和 t 是给定的两个串，在主串 s 中寻找等于子串 t 的部分的过程称为模式匹配， t 也称为模式。
- 如果在 s 中找到等于 t 的子串，则称匹配成功，返回 t 在 s 中的首次出现的存储位置，否则匹配失败。
- 模式匹配是字符串最核心的操作，广泛用于文本查找、数据过滤、搜索引擎
- 例：

$s = \text{"ababcabcacbab"}$

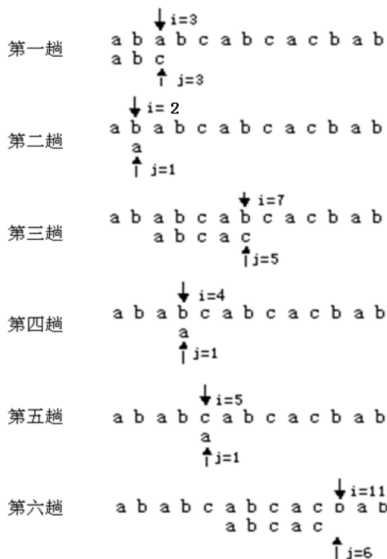
$t = \text{"abcac"}$

简单的模式匹配

s="ababcabcacbab"

t="abcac"

- Brute-Force 算法 (暴力匹配)
- 核心思想：逐位比较，失配则回溯
- 直观易懂，但效率低下



Python 示例代码 I

```
1 def bf_search(source, target) -> tuple:
2     """
3     从 source 中查找出现的 target, 返回第一个位置的二元组
4     主串中的位置, 比较次数), 未找到返回 (-1, 比较次数)
5     """
6     compare_count = 0
7     for i in range(len(source)):
8         found = True
9         for j in range(len(target)):
10             # 防止索引越界
11             if i + j >= len(source):
12                 found = False
13                 break
```

Python 示例代码 II

```
14         compare_count = compare_count + 1
15         if source[i+j] != target[j]:
16             found = False
17             break
18         if found:
19             return i, compare_count
20     return -1, compare_count
21
22 source = 'aaaaaaaaaab'
23 target = 'aaaab'
24 position, compare_count = bf_search(source, target)
25
26 print(' 返回位置: ', position)
27 print(f'在{source}的第{position}个位置开始, 发现了{target},
    ↪ 共比较了{compare_count}次')
```


Python 示例代码 III

```
28 print(source)
29 print('*'*position, end='')
30 print(target)
```

1 返回位置: 7

2 在aaaaaaaaaaab的第7个位置开始, 发现了aaaab, 共比较了40次

3 aaaaaaaaaaaab

4 *****aaaab

BF 算法时间复杂度分析

- 设串 s 长度为 n ，串 t 长度为 m 。匹配成功的情况下，考虑两种极端情况。
 - 在最好情况下，即每趟不成功的匹配都发生在第一对字符比较时。
 - 例如： $s = \text{"aaaaaaaaaabc"}$ ， $t = \text{"bc"}$ ，设匹配成功发生在 s_i 处。
 - 在前面 $i-1$ 趟不成功的匹配中共比较了 $i-1$ 次，第 i 趟成功的匹配共比较了 m 次，所以总共比较了 $i-1+m$ 次。
 - 所有匹配成功的可能共有 $n-m+1$ 种，设从 s_i 开始与 t 串匹配成功的概率为 p_i ，在等概率情况下 $p_i = 1/(n-m+1)$ ，因此最好情况下平均比较的次数是：

$$\sum_{i=1}^{n-m+1} p_i \times (i-1+m) = \sum_{i=1}^{n-m+1} \frac{1}{n-m+1} \times (i-1+m) = \frac{(n+m)}{2}$$

- 即最好情况下的时间复杂度是 $O(n+m)$

BF 算法时间复杂度分析

- 在最坏情况下，即每趟不成功的匹配都发生在 t 的最后一个字符。
- 例如： $s = \text{"aaaaaaaaaab"}$ ， $t = \text{"aaab"}$ ，设匹配成功发生在 s_i 处。
- 在前面 $i - 1$ 趟匹配中共比较了 $(i - 1) \times m$ 次，第 i 趟成功的匹配共比较了 m 次，所以总共比较了 $i \times m$ 次，因此最坏的情况下平均比较的次数是：

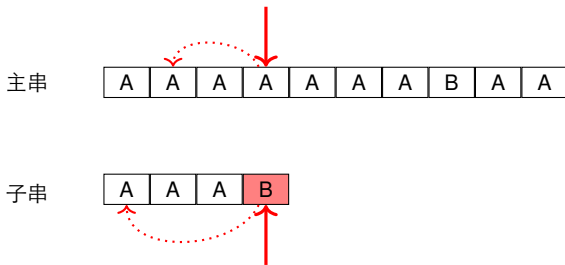
$$\sum_{i=1}^{n-m+1} p_i \times (i \times m) = \sum_{i=1}^{n-m+1} \frac{1}{n-m+1} \times (i \times m) = \frac{m \times (n-m+2)}{2}$$

- 即最坏情况下的时间复杂度是 $O(n \times m)$

为什么 BF 算法时间性能低？

在每趟匹配不成功时存在大量回溯 (每次移动一位开始新的比较)，没有利用已匹配信息，造成重复比较

改进算法



- 核心思想：不回溯主串指针，利用已匹配信息，让模式串多滑动几位
- 改进算法的目的是在每一趟匹配过程中出现不匹配时，向右“滑动”尽可能远的一段距离后，继续进行比较。
- 经典高效算法：
 - KMP 算法: 由 D. E. Knuth, J. H. Morris 和 V. R. Pratt 同时发现
 - Boyer-Moore 算法（实际工程中常用）

KMP 算法 I

- 核心思想：利用已经得到的“部分匹配”的结果将模式向右滑动，避免主串回溯，将时间复杂度优化至 $O(n + m)$
 - ① 当匹配失败时，看看模式串前面有没有重复的“头尾”
 - ② 有的话，直接把模式串跳到能接上的位置，不用从头比
 - ③ 例子：模式：A B C A B C D 匹配到 D 失败，但前面匹配了 A B C
 - ④ 发现模式串前缀 A B C = 后缀 A B C
 - ⑤ $\Rightarrow$ 直接把模式串跳过去，主串不动
- 关键结构：**next** 数组——记录模式串的最长相等前后缀，决定失配后滑动的位置
- 优势：处理长文本、重复字符多的场景时，效率远高于 BF 算法

KMP 算法 II

视频讲解：

- <https://www.bilibili.com/video/BV1AY4y157yL/>
- <https://www.bilibili.com/video/BV1Er421K7kF/>

KMP 算法

- 算法流程：
 - ① 预处理模式串，构建 next 数组
 - ② 主串与模式串指针均不回溯
 - ③ 失配时，模式串根据 next 数组滑动
 - ④ 匹配成功则返回位置
- 代码实现：利用大语言模型来完成代码，已经变得非常方便，参见 `string.ipynb` 文件。
- 应用场景：单一模式串、长文本高效匹配

课堂练习

- 任务：从《红楼梦》中，统计主要人物出现的次数，并按照频次高低输出。
- 输入数据：
 - 文本文件：ipynb/honglou.txt
 - 人物列表：ipynb/honglou_person.txt
- 实现思路：
 - 读取文本与人物列表
 - 使用字符串匹配算法查找人物
 - 统计频次并排序输出
- 进阶思考：人物数量成百上千时，用 KMP 逐一遍历效率低，如何优化？

多模式串匹配与 AC 自动机

- 单模式匹配 VS 多模式匹配：
 - BM、KMP 为单模匹配：一次只匹配一个模式串
 - 多模式匹配：同时匹配成千上万个模式串
- 假设主串 $T[1, \dots, m]$ ，模式串有 k 个 $P = P_1, \dots, P_k$ ，总长度为 n 。如果采用 KMP 来匹配多模式串，则算法复杂度为：

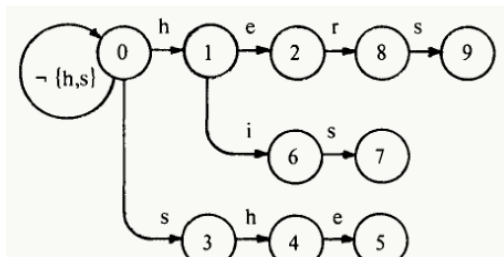
$$O(|P_1| + m + \dots + |P_k| + m) = O(n + km)$$

- KMP 缺陷：没有利用模式串之间的重复字符结构，主串需要重复扫描 k 次
- 解决方案：**AC 自动机 (Aho-Corasick)** 贝尔实验室的 Aho 与 Corasick 于 1975 年基于有限状态机提出¹

¹AC 算法比 KMP 提出要早，KMP 于 1977 年提出

AC 自动机核心结构

- AC 自动机 = Trie 树（前缀树）+ 失败指针 + 输出函数
- 三大核心函数：
 - **goto**: 字符匹配成功的状态转移
 - **failure**: 匹配失败时的回退状态（类似 KMP 的 next 数组）
 - **output**: 匹配成功时输出结果
- 示例模式串: {he, his, hers, she}
- goto 函数: 利用模式串公共前缀构建 Trie 树



AC 自动机

- **failure** 函数：匹配失败时，跳转到最长可匹配后缀节点，避免从头匹配

i	1	2	3	4	5	6	7	8	9
$f(i)$	0	0	0	1	2	0	3	0	3

(b) Failure function.

AC 自动机

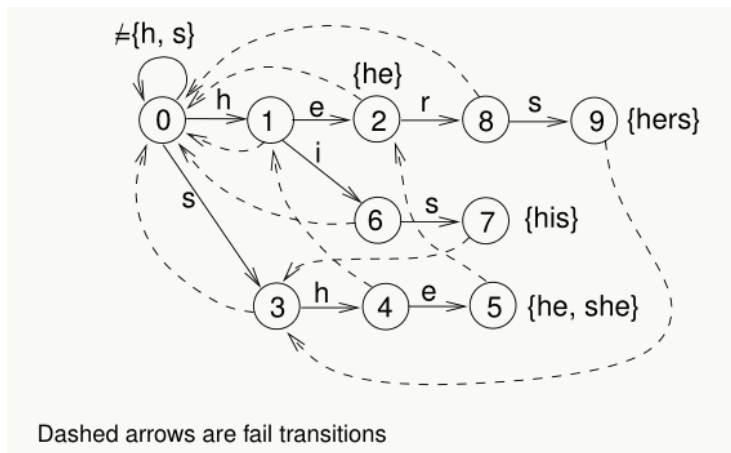
- output 函数：记录每个终止状态对应的匹配成功的模式串

<i>i</i>	<i>output(i)</i>
2	{he}
5	{she, he}
7	{his}
9	{hers}

(c) Output function.

AC 自动机

- 完整的 AC 自动机：整合 goto、failure、output，一次遍历完成所有模式串匹配



AC 自动机匹配过程

AC 算法根据自动机匹配模式串，过程简单高效：从主串的首字符、自动机的初始状态 0 开始遍历

- 若字符匹配成功：按 goto 函数转移到下一状态；若当前状态有 output，则输出匹配结果
- 若字符匹配失败：递归按 failure 函数转移，无需回溯主串
- 最终复杂度： $O(n + m + z)$ ， n = 主串长度， m = 模式串总长度， z = 匹配结果数
- 优势：海量关键词匹配场景下，性能远超循环 KMP

Python 实战: pyahocorasick

- 工业级 AC 自动机库: 底层 C 实现, 速度极快
- 官方地址: <https://pypi.org/project/pyahocorasick/>
- 教学视频:
<https://www.bilibili.com/video/BV1yA4y1Z74t/>
- 核心用法:
 - ① 构建自动机
 - ② 添加所有关键词
 - ③ 生成 AC 自动机
 - ④ 扫描文本, 一次性获取所有匹配结果
- 典型应用: 敏感词过滤、人物统计、日志检索、生物信息分析

字符串匹配算法总结

- BF 算法：简单易懂，效率低， $O(nm)$ ，适合小规模数据
- KMP 算法：无回溯， $O(n + m)$ ，适合单模式串匹配
- AC 自动机：Trie+ 失败指针， $O(n + m + z)$ ，适合海量多模式串匹配
- 工程选择：
 - 关键词少：BF/KMP
 - 关键词成千上万：AC 自动机 (pyahocorasick)

学习重点

理解算法思想 + 掌握 Python 工具库 + 能解决实际文本处理问题